
USING LOW-RANKING ADAPTATION TO FINETUNE THE SEGMENT ANYTHING MODEL FOR CUSTOM APPLICATIONS

Van Eetvelde Arno

Remouchamps Nick

Coretecs

{arno.vaneetvelde, nick.remouchamps}@coretecs.be

ABSTRACT

Large language models (LLMs) have demonstrated remarkable advancements by fine-tuning pre-trained models with specific, smaller datasets. Inspired by this, we explored a similar approach for computer vision tasks, focusing on fine-tuning the Segment Anything Model (SAM) for segmenting industrial electrical components within cabinets. This adaptation aims to improve the inspection process for our Electrical and Instrumentation (E&I) team. We chose Low-Rank Adaptation (LoRA) due to its efficiency in requiring minimal data and computational resources. Our study involved creating a dataset of 100 annotated images, fine-tuning SAM models of various sizes, and evaluating their performance. We also developed a custom mask generator to address inconsistencies in the default mask generator provided by FAIR, enhancing the segmentation accuracy for our specific use case. This paper presents the methodology, experiments, and results of fine-tuning SAM using LoRA, demonstrating its effectiveness and potential for practical applications in industrial settings. Future work will focus on expanding the system's capabilities to classify segmented objects and further refining the workflow for diverse applications.

1 INTRODUCTION

Large language models (LLMs) have recently seen a significant increase in popularity and functionality. A common approach involves using a large LLM trained by a major corporation on extensive datasets and fine-tuning it for specific tasks using smaller, more focused datasets. Inspired by this trend, we aimed to apply a similar methodology to computer vision tasks, specifically focusing on segmenting parts of an image following the release of the Segment Anything Model (SAM) by Meta AI Research (FAIR).

To make this concept more tangible, we explored specific use cases and decided to develop a system capable of detecting industrial electrical components within an industrial cabinet. This system could potentially streamline the inspection process for our internal Electrical and Instrumentation (E&I) team. However, this paper will concentrate on the technical and theoretical aspects of fine-tuning the SAM. A more comprehensive article detailing the entire application and pipeline for the E&I use case will be published separately.

After evaluating various methods for fine-tuning large models, we chose Low-Rank Adaptation (LoRA) due to its minimal data and resource requirements. Initial tests with a small number of images led us to create a dataset of 100 annotated images, which we used to fine-tune models of different sizes with varying hyperparameters. The results of these experiments are presented in the analysis section.

In addition to fine-tuning SAM, we developed a custom mask generator to work in conjunction with the fine-tuned model, specifically to extract all electrical components from an image. This was necessary because the default mask generator provided by FAIR was inconsistent for our use case. The details of this custom solution are discussed in the solution section of the paper.

2 LITERATURE REVIEW

In this section, we will briefly explain the two primary papers that informed our project and discuss why we selected these architectures. For a deeper understanding, we recommend consulting the original, highly insightful papers.

2.1 SEGMENT ANYTHING

The Segment Anything paper (Kirillov et al., 2023), released by Meta AI Research (FAIR), introduces a pioneering computer vision foundation model. The paper presents four key contributions: a new computer vision task, a new foundation model architecture, a novel data engine, and a new instance segmentation dataset. Our focus is primarily on the new foundation model architecture, as this is the model we have fine-tuned and utilized.

The proposed Segment Anything Model (SAM) comprises three main components, illustrated in Figure 1: an image encoder, a flexible prompt encoder, and a fast mask decoder. The image encoder is responsible for extracting image embeddings, which is the most computationally intensive part of the model. However, once the image embeddings are extracted, they can be reused with different prompts to generate multiple outputs, making it ideal for images containing numerous objects.

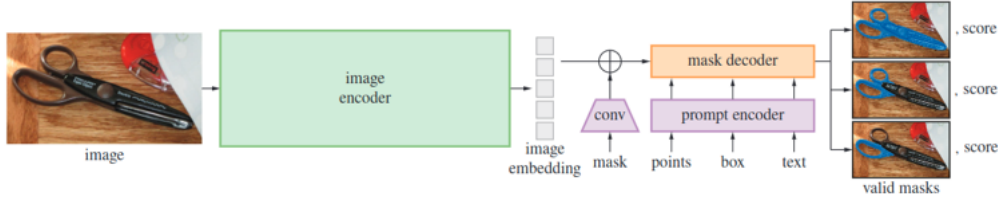


Figure 1: Segment Anything Model (SAM) overview. A heavyweight image encoder outputs an image embedding that can then be efficiently queried by a variety of input prompts to produce object masks at amortized real-time speed. For ambiguous prompts corresponding to more than one object, SAM can output multiple valid masks and associated confidence scores.

An ambiguity problem arises when a specific prompt can correspond to multiple masks. To address this, the mask decoder outputs three possible masks, each with a confidence score (estimated intersection over union).

FAIR has released three pre-trained models based on different sized Transformer vision models: ViT-B, ViT-L, and ViT-H, with 91 million, 308 million, and 636 million parameters, respectively. These models were trained on the SA-1B dataset, which consists of 11 million licensed and privacy-preserving images with over 1 billion masks.

2.2 LOW-RANKING ADAPTATION

Low-Rank Adaptation (LoRA) is an innovative fine-tuning technique developed to address the challenges of training large-scale models with limited data and computational resources (Hu et al., 2021). Traditional fine-tuning methods can be resource-intensive and require large amounts of labeled data, which is often impractical. LoRA, by contrast, introduces a more efficient approach by focusing on adapting only a subset of model parameters.

LoRA operates by freezing the pretrained weights during training and introducing a decomposition layer in parallel, as illustrated in Figure 2. This technique enables a more compact representation and reduces the number of parameters that need to be fine-tuned.

LoRA has been shown to maintain or even improve model performance while reducing the computational burden. This makes it

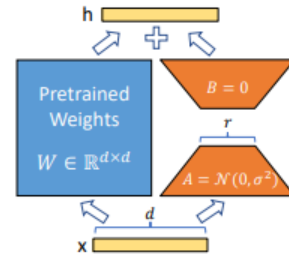


Figure 2: LoRA reparametrization. We only train A and B.

an attractive option for scenarios where resources are limited but high model accuracy is still required.

In our study, we chose LoRA for fine-tuning the Segment Anything Model (SAM) due to its efficiency and effectiveness. By leveraging LoRA, we aimed to adapt SAM to our specific task of segmenting industrial electrical components, despite having a relatively small annotated dataset. The use of LoRA enabled us to achieve satisfactory results with reduced computational resources, validating its suitability for fine-tuning large models in practical applications.

3 PROBLEM STATEMENT

Current classical methods for instance segmentation require training models on specific parts. Consequently, when presented with an unseen part, these models behave unpredictably; they may detect it if it looks similar enough to the training data, but often they do not. Our goal is to train a model capable of segmenting all important parts, even those not seen during training. Once the parts are segmented, we can later apply a feature extractor combined with a clustering algorithm to classify the objects. However, this paper will focus solely on the segmentation component of this pipeline.

4 ANALYSIS

In this section, we will discuss our approach and the results of our experiments. Specifically, we will begin by detailing the datasets we created and used for fine-tuning the model. Following this, we will explain the training methodology employed. Finally, we will present and analyze the results of our fine-tuning efforts.

4.1 DATA

After conducting initial tests on a dummy dataset of 10 training images (with no validation images), we proceeded to create our main dataset. This dataset consists of 100 annotated images, divided into 85 training images and 15 validation images. Additionally, 20 unannotated images were used as the testing dataset. All images were annotated using CVAT, in collaboration with our E&I team, to ensure the highest possible accuracy.

The dataset contains a total of 1,292 annotated objects, averaging 13 objects per image. Specifically, the training set includes 1,067 instances, while the validation set contains 225 objects (Table 1). Below are a few examples of the annotated images (Figure 4), along with the segmentation area distribution for both the training and validation datasets (Figure 3).

Task	Number of images	Number of annotated objects
Training	85	1067
Validation	15	225
Testing	20	/
Total	120	1292

Table 1: Dataset segmentation statistics per task.

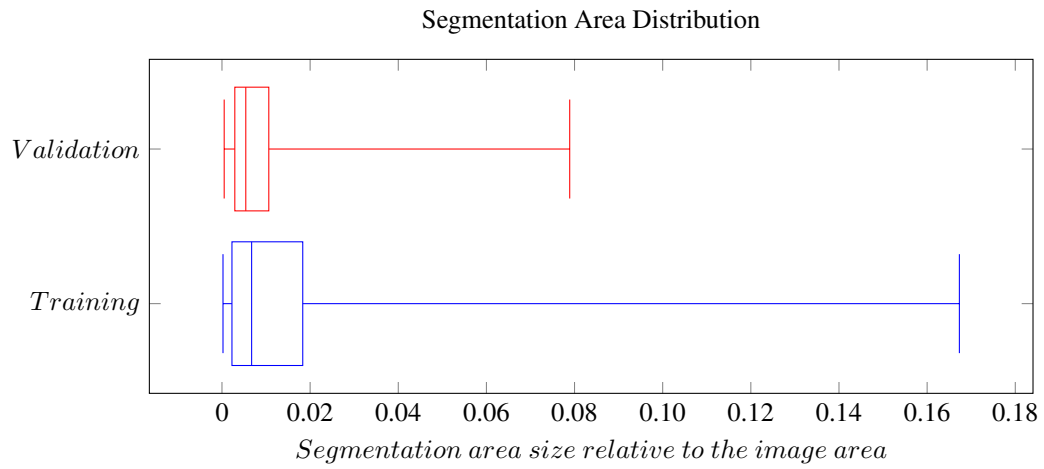


Figure 3: Box-plot of the area distribution of the validation and training dataset.



Figure 4: Example of annotated images from the training dataset. Each colored area is a separate mask that was manually annotated.

4.2 TRAINING

To fine-tune the large Segment Anything models, we employed a technique called Low-Rank Adaptation (LoRA). This approach was chosen because it requires less data and computational resources than traditional fine-tuning methods.

We used dice loss for loss calculations during training and validation. However, to evaluate the models on the unannotated test set, we manually counted the number of correctly segmented objects using the `SamAutomaticMaskGenerator` class provided by FAIR. This manual counting approach

reflects the model’s practical application and provides a more accurate representation of its performance. (Note: We only checked for correctly detected objects, not incorrectly detected ones.)

For the actual LoRA fine-tuning, we heavily adapted the code from <https://github.com/WangRongsheng/SAM-fine-tune> and rented GPUs from <https://lambdalabs.com/>. We utilized <https://www.tensorflow.org/tensorboard> for our MLOps workflow and to compare results.

After setting up the environment and defining the training and testing protocols, we began by training and testing the smallest SAM model with different hyperparameters. Based on these results, we moved on to the medium and large SAM models. The checkpoint with the lowest validation loss was used as the final model for testing. All result metrics are based on this model.

Disclaimer It would likely have been more accurate to compare the models using an annotated test set and evaluating the IoU of the generated masks against the ground truth masks, as this aligns with how the models were trained. Additionally, it’s important to recognize that the segmentation results are highly dependent on the prompt generation within the SamAutomaticMaskGenerator class. This is why our solution section discusses our custom mask generator approach to enhance the final results. However, while comparing the fine-tuned SAM models, we used the default SamAutomaticMaskGenerator class provided by FAIR with its default settings.

This realization came a bit too late, and re-evaluating all the models would have been too time-consuming and labor-intensive. For our purposes, this is not a significant issue, as detecting most objects suffices for our application.

4.3 RESULTS

As previously mentioned, we began training and testing with the smallest SAM model to facilitate quick revisions and to identify optimal hyperparameters before scaling up to the larger models, which require significantly more time and resources. The parameter counts of the Segment Anything models are displayed in [Table 2](#) using a LoRA rank of 512.

Model	Number of parameters in the base model	Number of trainable LoRA parameters	Total number of parameters
SAM ViT-B	91M	23M	113M
SAM ViT-L	308M	54M	363M
SAM ViT-H	636M	88M	725M

Table 2: Model parameter counts when using a LoRA rank of 512.

4.3.1 NO TRAINING MODEL COMPARISON

The initial step was to compare the three models without any fine-tuning. As shown in [Table 3](#) and [Figure 5](#), the performance of the models improved with their size. However, the performance increase from the medium model to the large model was noticeably smaller compared to the increase from the small model to the medium model.

Model	Accuracy on test set
SAM ViT-B	8%
SAM ViT-L	28%
SAM ViT-H	31%

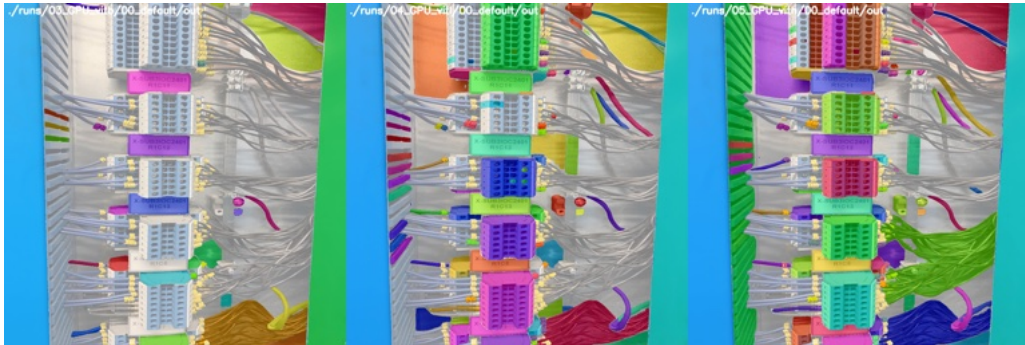
Table 3: Accuracy on test set without any fine-tuning



(a) Comparing image 00 from the test set.



(b) Comparing image 05 from the test set.



(c) Comparing image 16 from the test set.

Figure 5: Results of the three non fine-tuned models on three images from the test set using the default SamAutomaticMaskGenerator from FAIR. The leftmost image is from SAM ViT-B, the middle image is from SAM ViT-L, and the rightmost image is from SAM ViT-H. Each colored area represents a separate mask generated by the model and mask generator.

4.3.2 SAM ViT-B

During the training and testing of the smallest model, our primary objectives were to determine the necessary variation in point generation for optimal results and to identify the appropriate rank for our use-case. When training our model, we generate a point within the ground truth mask to use as the input prompt. The allowable distance this point can move from the center of the mask is referred to as point variation. Specifically, a point variation of 0.5 means the point can shift up to 25% of the bounding box’s width to the left or right (and similarly for the height).

All tests were conducted over 20 epochs with a batch size of 4 images on a GPU provided by Lambda Labs, the results can be found in [Table 4](#)

Experiment	Base model	LoRA rank	Point variation	Accuracy on test set
00 (default model)	ViT-B	/	/	8%
01	ViT-B	512	0.20	61%
02	ViT-B	1024	0.20	63%
03	ViT-B	512	0.5	65%
04	ViT-B	1024	0.5	60%
05	ViT-B	512	1.00	52%

Table 4: Accuracy on test set using the SAM ViT-B model as base.

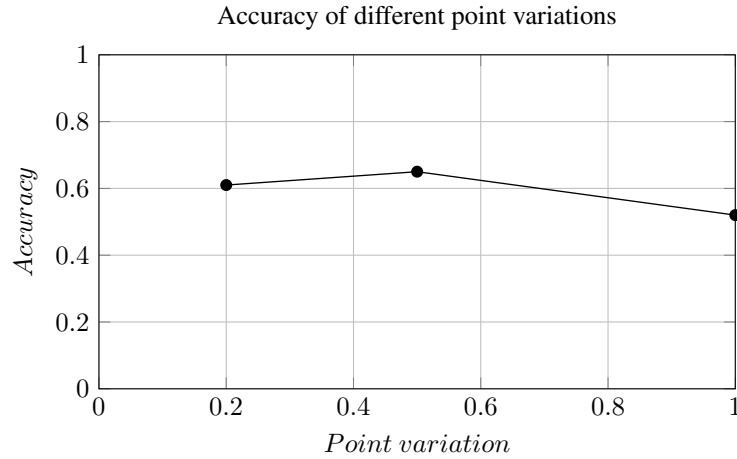


Figure 6: Accuracy on the test set using the SAM ViT-B model as base and a LoRA rank of 512 with different point variations.

Our results indicate that LoRA training significantly improves the number of correctly detected objects, as expected. Additionally, a point variation of 1.00 was found to be excessive, while 0.50 appeared to be the sweet spot (Figure 6). Figure 7 and Figure 8 show two comparisons of the six experiments on the same image.



Figure 7: A comparison of the six experiments done on SAM ViT-B using the default SamAutomaticMaskGenerator for inference on image 10 from the test set. From [Table 4](#): upper-left is experiment 00, upper-middle is experiment 01, upper-right is experiment 02, lower-left is experiment 03, lower-middle is experiment 04, and lower-right is experiment 05.

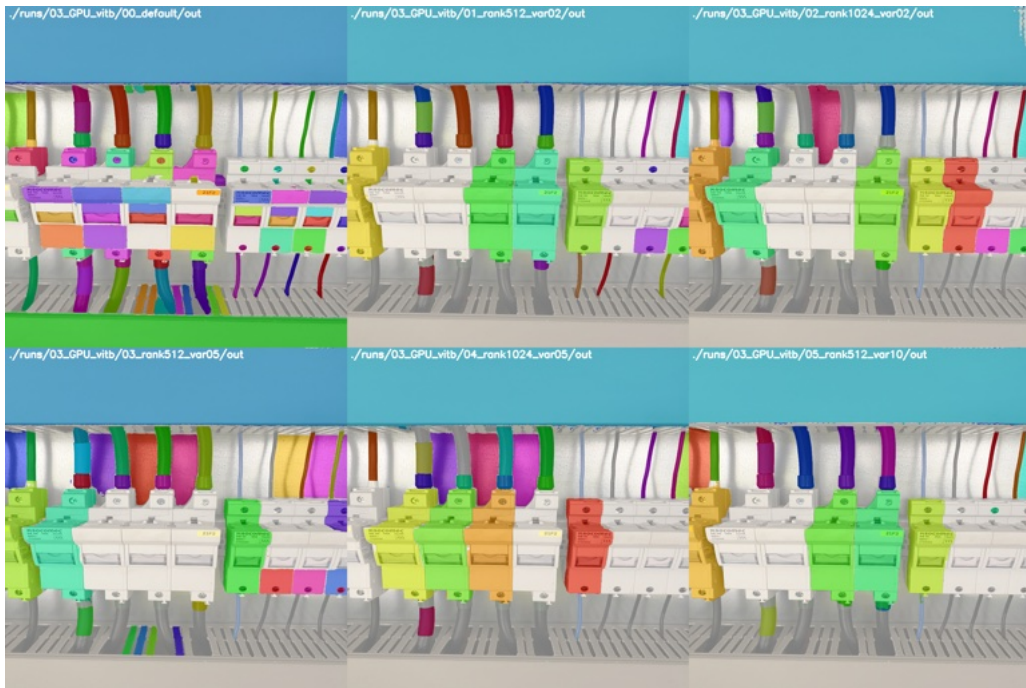


Figure 8: A comparison of the six experiments done on SAM ViT-B using the default SamAutomaticMaskGenerator for inference on image 18 from the test set. From [Table 4](#): upper-left is experiment 00, upper-middle is experiment 01, upper-right is experiment 02, lower-left is experiment 03, lower-middle is experiment 04, and lower-right is experiment 05.

4.3.3 SAM ViT-L

When comparing different ranks on the medium model, we observe similar results to those of the small model. This leads us to conclude that a rank of 512 appears to be a suitable middle ground for our use case. [Figure 9](#) illustrates the accuracy of the different LoRA ranks with the point variation set to 0.5.

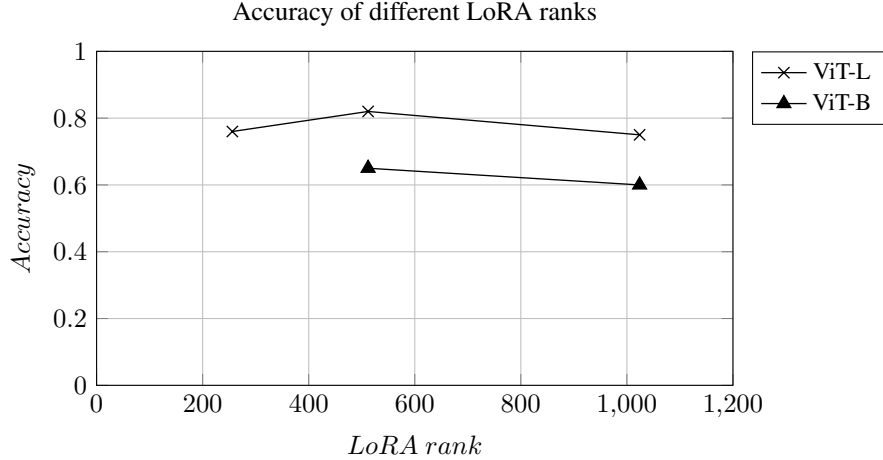


Figure 9: Accuracy on the test set comparing the different SAM base models and LoRA ranks with the point variation set to 0.5.

4.3.4 SAM ViT-H

The largest SAM ViT-H model was trained once with a point variation of 0.5 and a LoRA rank of 512, which yielded the best results among the smaller-sized models. Due to the substantial time and computational resources required, only a single run of 20 epochs was conducted on the larger model architecture.

Upon evaluating the fine-tuned model using the same metric as before (manually counting the correctly segmented parts using the default `SamAutomaticMaskGenerator`) on the test set, we observed that it performed worse than the fine-tuned medium model, achieving an accuracy of 66%. This discrepancy may be attributed to overfitting on the training and validation datasets, although further investigation is warranted in the future.

Ultimately, we concluded that the performance of the medium model was more than sufficient for our use case, and the longer inference time of the larger model would be undesirable.

5 SOLUTION

Our solution primarily involves the procedure to fine-tune the Segment Anything models. We discovered that with 100 well-annotated images, it is possible to fine-tune the large SAM models using LoRA techniques. Specifically, a LoRA rank of 512 with a point variation of 0.5 provided the best results on the medium model.

5.1 MASK GENERATOR

With our large, fine-tuned segmentation model capable of segmenting electrical components with a prompt, the goal is to provide an image and receive all segmented parts automatically. Normally, this would require manually providing points, bounding boxes, or rough masks. To automate this process, FAIR offers a `SamAutomaticMaskGenerator` class in their GitHub repository. This class generates points based on input parameters and post-processes the resulting masks.

During our testing, we found the SamAutomaticMaskGenerator from FAIR underperformed in our specific use case. However, when we manually placed a point on an undetected part, the model successfully segmented it. This observation motivated us to develop a custom mask generator tailored to our needs.

Our custom mask generator involves the following steps (also showed in [Figure 10](#)):

1. Initial Mask Generation:

- (a) Generate masks from a 16x16 grid of evenly distributed points, totalling 256 points.
- (b) Use the SAM model to generate three masks per point, retaining only the first two (the largest mask is typically a background mask and not useful for our purpose).

2. Mask Refinement:

- (a) Refine the generated masks by recalculating each mask from its center point, ensuring masks are generated from their center points to avoid intersecting multiple objects.
- (b) Filter the masks by discarding any that intersect more than 20% (using the Intersection over Union metric) with another mask having a higher predicted IoU score from the SAM model.
- (c) These two steps are repeated three times.

3. Additional Point Generation:

- (a) Generate a new grid of points on all undetected parts of the image to capture any missed objects. This reduces the number of masks to generate, as we only run the model on the undetected areas, thereby speeding up the inference process.
- (b) Run the model again on the new grid of points, generating masks and retaining only the first two masks per point.

4. Refinement and Filtering of New Masks:

- (a) Refine and filter these new masks using the same steps as previously described under Mask Refinement, repeating the process three times.

5. Final Mask Consolidation:

- (a) Concatenate the two sets of generated masks and filter out overlapping masks with an IoU higher than 20%, keeping the mask with the highest predicted IoU score from the SAM model.

This process ensures a comprehensive and accurate segmentation of all electrical components in the image, leveraging both initial and refined mask generation techniques. The inference time on a CPU on our test set was on average 49.66 seconds when using our custom mask generator with the medium SAM model and a LoRA rank of 512. A few example images on our test set can be found in [Appendix A](#).

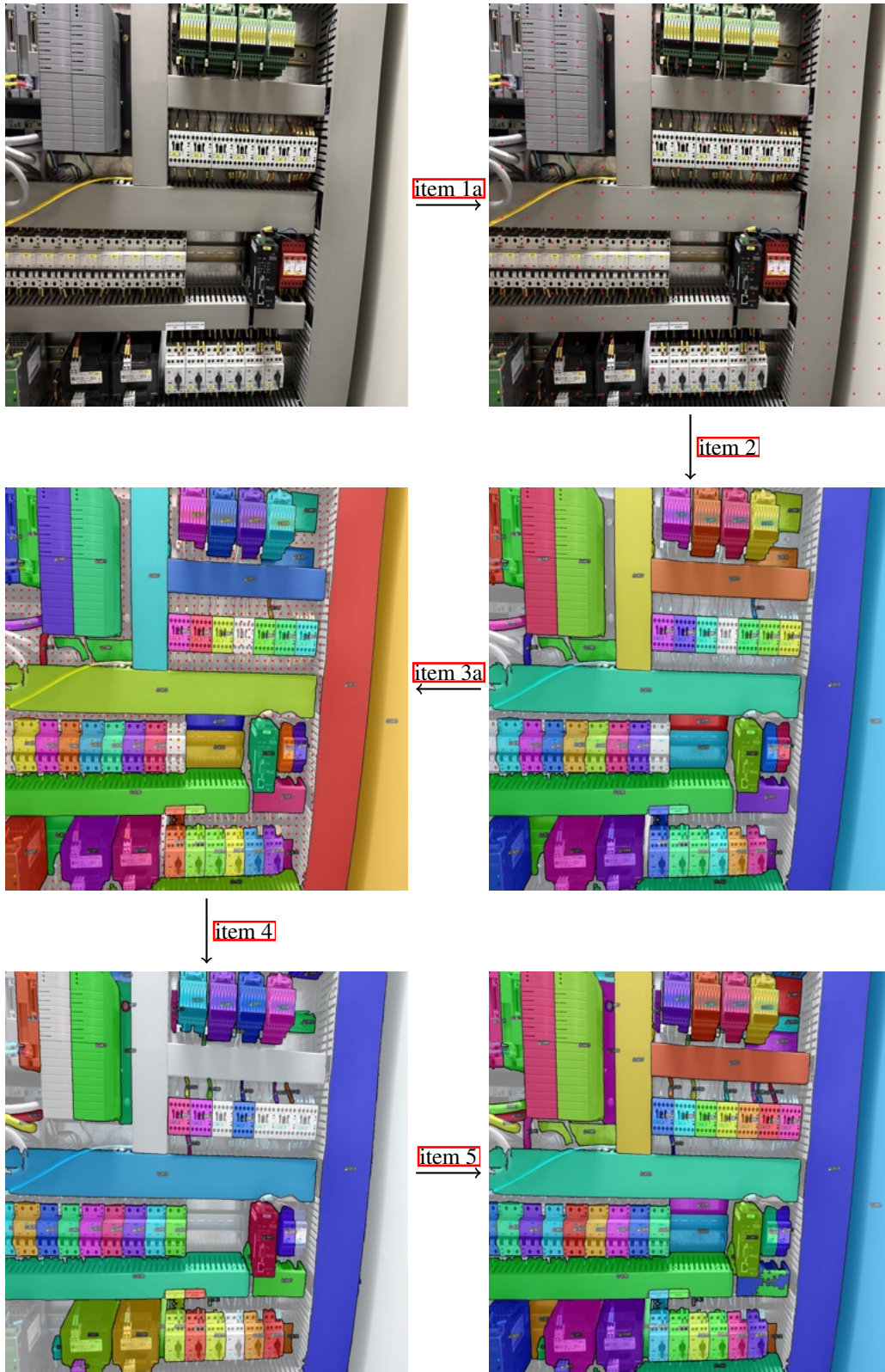


Figure 10: Custom maks generator pipeline with references to the steps defined in [Section 5.1](#)

6 CONCLUSION

In conclusion, fine-tuning a large pretrained base model using LoRA techniques, as is currently popular with LLMs, appears to be a viable solution for image segmentation tasks. This approach generalizes well and, when combined with an effective prompt generator (like our custom mask generator), it can be a powerful system for custom use cases.

We will continue to research and develop our custom use case, building on this robust general object segmentation model. Our goal is to create a system that can also classify segmented objects and provide our E&I team with valuable additional information.

Future work should focus on researching and developing the workflow and architecture across different use cases. Additionally, investigating why the larger model performed worse than the medium model would be beneficial.

7 ACKNOWLEDGEMENTS

I would like to take this opportunity to thank Coreteks for providing the opportunity to conduct this research and for funding the GPU rentals. I am also grateful to the people at Coreteks who assisted in the development process. In particular, I would like to thank Nick Remouchamps for his supervision and useful insights. Additionally, I extend my thanks to Wouter Van de Wiele for providing high-quality pictures, and to Simon De la Marche for his valuable input in the annotation process.

REFERENCES

Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models, 2021.

Alexander Kirillov, Eric Mintun, Nikhila Ravi, Hanzi Mao, Chloe Rolland, Laura Gustafson, Tete Xiao, Spencer Whitehead, Alexander C. Berg, Wan-Yen Lo, Piotr Dollár, and Ross Girshick. Segment anything, 2023.

A SOLUTION ON TEST SET IMAGES

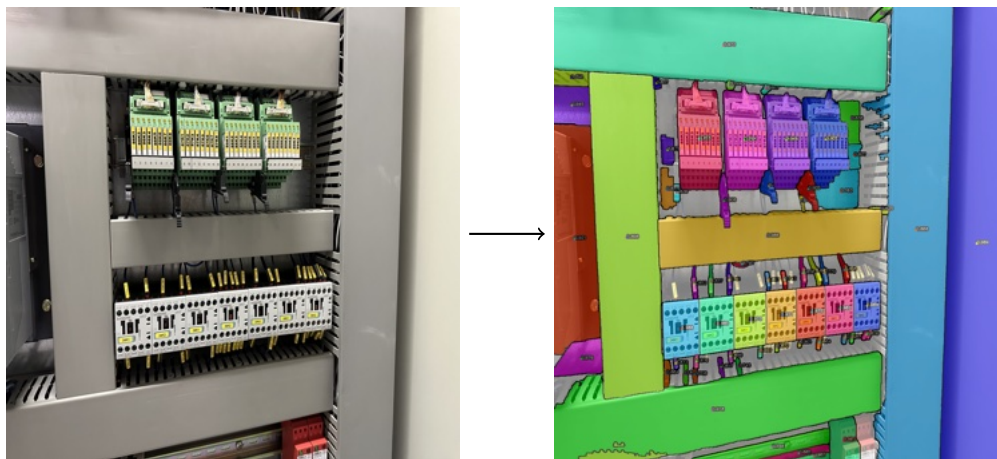


Figure 11: IMG_0491



Figure 12: IMG_0530

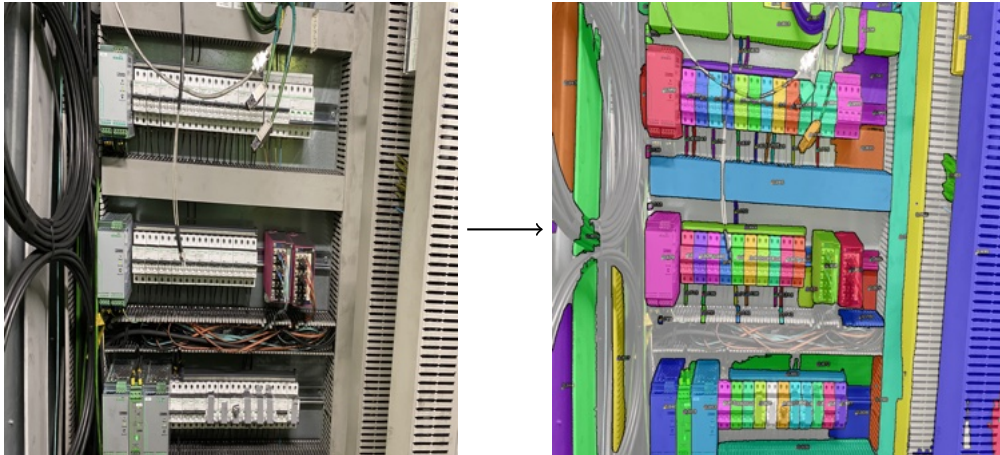


Figure 13: IMG_1185

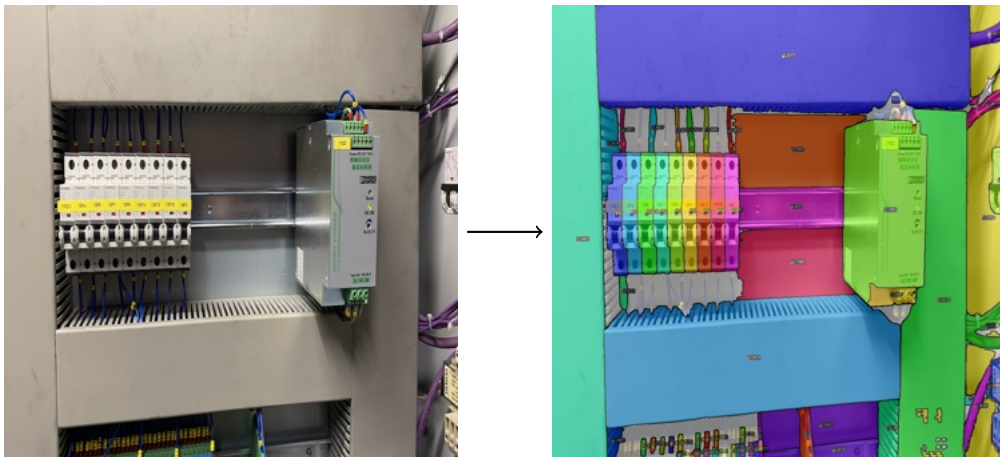


Figure 14: IMG_1249

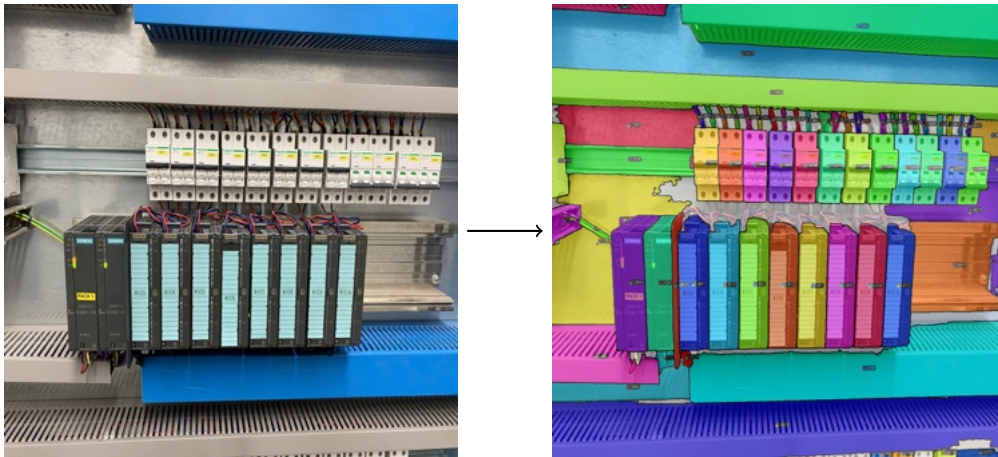


Figure 15: IMG_1753

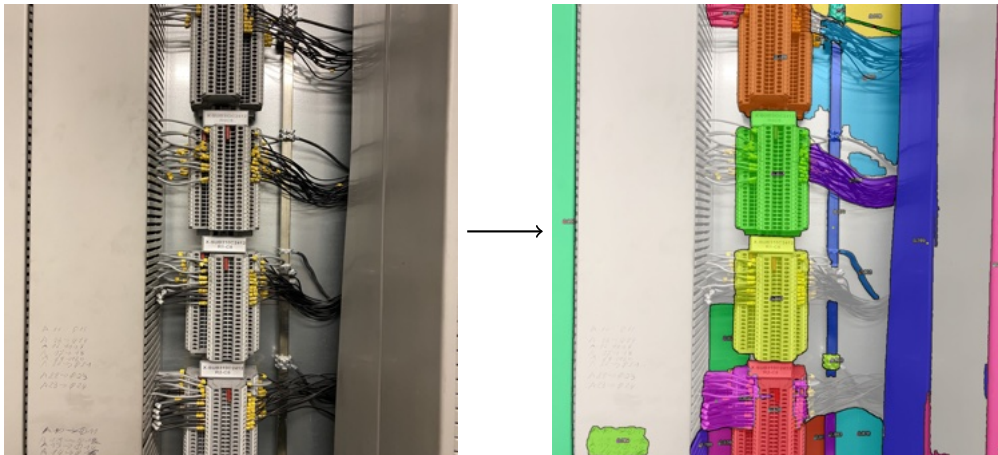


Figure 16: IMG_1801

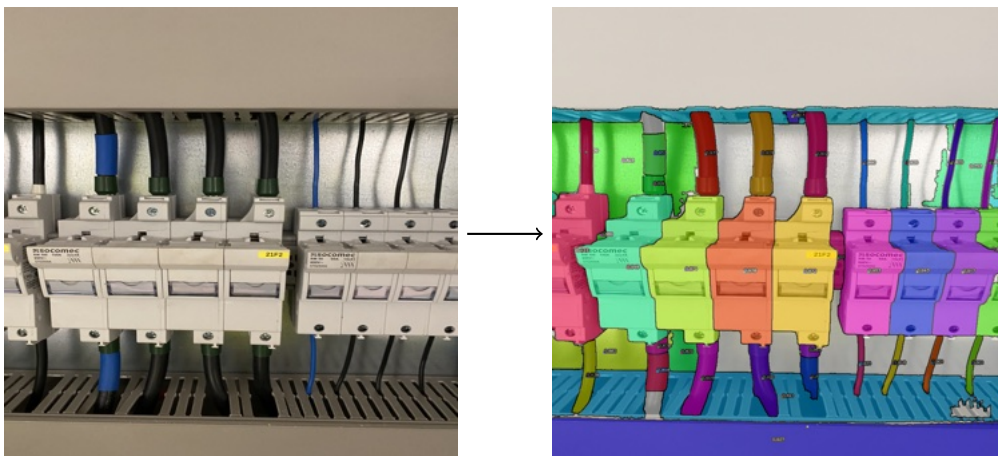


Figure 17: IMG_2514

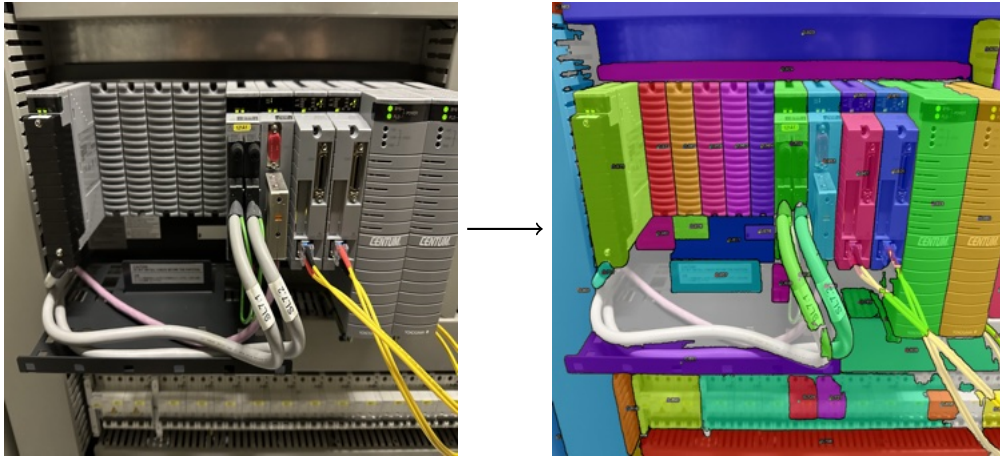


Figure 18: IMG_2535